프로젝트 구조 분석 (Project Structure Analysis)

개요

이 문서는 Monkey 프로그래밍 언어 인터프리터/컴파일러 프로젝트의 전체 구조를 분석한 결과입니다.

프로젝트 디렉토리 구조

```
monkey-go/
├── ast/ # 추상 구문 트리 (AST) 정의
| ├── ast.go # Node, Statement, Expression 인터페이스 및 구체적 노드들
| └── ast_test.go # AST 테스트
├── benchmark/ # 성능 벤치마크
| └── main.go # 벤치마크 실행 코드
├── code/ # 바이트코드 정의 및 처리
| ├── code.go # Opcode 상수, Instructions 타입, Make/Lookup 함수
| └── code_test.go # 바이트코드 테스트
├── compiler/ # 컴파일러 구현
| ├── compiler.go # Compiler 구조체, Compile() 함수, 바이트코드 생성
| ├── compiler_test.go # 컴파일러 테스트
| ├── symbol_table.go # SymbolTable 구조체, 심볼 관리
```

```
| └── symbol_table_test.go # 심볼 테이블 테스트
├── evaluator/ # 인터프리터 평가기
| ├── builtins.go # 내장 함수 정의 (len, first, last, rest, push)
| ├── evaluator.go # Eval() 함수, 표현식/문장 평가 로직
| └── evaluator_test.go # 평가기 테스트
├── lexer/ # 어휘 분석기 (렉서)
| ├── lexer.go # Lexer 구조체, NextToken() 함수
| └── lexer_test.go # 렉서 테스트
├── object/ # 객체 시스템
| ├── builtins.go # 내장 함수 객체 정의
| ├── environment.go # Environment 구조체, 변수 바인딩 관리
| ├── object.go # Object 인터페이스, 각종 객체 타입들
| └── object_test.go # 객체 시스템 테스트
├── parser/ # 구문 분석기 (파서)
| ├── parser.go # Parser 구조체, Pratt Parser 구현
| └── parser_test.go # 파서 테스트
├── repl/ # Read-Eval-Print Loop
| └── repl.go # Start() 함수, 대화형 인터페이스
├── token/ # 토큰 정의
| └── token.go # TokenType 상수, Token 구조체, 키워드 맵
├── vm/ # 가상 머신
| ├── frame.go # Frame 구조체, 함수 호출 프레임 관리
| ├── vm.go # VM 구조체, Run() 함수, 스택 기반 실행
```

```
│ └── vm_test.go # VM 테스트
├── main.go # 메인 진입점
└── README.md # 프로젝트 설명
```

핵심 컴포넌트 분석

1. 토큰화 (Tokenization) - lexer/, token/

주요 구조체:

- `Lexer` : 입력 문자열을 토큰으로 변환하는 구조체
- `input` : 입력 문자열
- `position` : 현재 위치
- `nextPosition` : 다음 위치
- `ch` : 현재 문자

주요 함수:

- `NextToken()` : 다음 토큰을 반환하는 핵심 함수
- `readChar()` : 다음 문자로 이동
- `peekChar()` : 다음 문자를 미리 보기
- `readIdentifier()` , `readNumber()` , `readString()` : 각 타입별 읽기

토큰 타입:

- 식별자/리터럴: `Identifier` , `Int` , `String`
- 연산자: `Plus` , `Minus` , `Asterisk` , `Slash` , `Equal` , `NotEqual`
- 구분자: `Comma` , `Semicolon` , `LeftParen` , `RightParen` 등
- 키워드: `Function` , `Let` , `If` , `Else` , `Return` , `True` , `False`

2. 구문 분석 (Parsing) - parser/, ast/

주요 구조체:

- `Parser` : Pratt Parser 구현체
- `prefixParseFns` : 전위 파싱 함수 맵
- `infixParseFns` : 중위 파싱 함수 맵
- `precedences` : 연산자 우선순위 맵

AST 노드 계층:

- `Node` : 기본 인터페이스 (`TokenLiteral()` , `String()`)
- `Statement` : 문장 노드 (`LetStatement` , `ReturnStatement` , `ExpressionStatement`)
- `Expression` : 표현식 노드 (`Identifier` , `IntegerLiteral` , `InfixExpression` 등)

주요 함수:

- `ParseProgram()` : 전체 프로그램 파싱
- `parseStatement()` : 문장 파싱
- `parseExpression()` : 표현식 파싱 (우선순위 기반)

3. 평가 (Evaluation) - evaluator/, object/

주요 구조체:

- `Environment` : 변수 바인딩과 스코프 관리
- 객체 타입들: `Integer` , `Boolean` , `String` , `Array` , `Hash` , `Function`

주요 함수:

- `Eval()` : AST 노드를 평가하여 객체 반환
- `evalProgram()` : 프로그램 전체 평가
- `evalInfixExpression()` : 중위 표현식 평가

- `applyFunction()` : 함수 호출 처리

내장 함수:

- `len()` : 길이 반환
- `first()` , `last()` : 배열 첫/마지막 요소
- `rest()` : 배열 나머지 요소
- `push()` : 배열에 요소 추가

4. 컴파일 (Compilation) - compiler/

주요 구조체:

- `Compiler` : AST를 바이트코드로 변환
- `instructions` : 생성된 명령어들
- `constants` : 상수 풀
- `symbolTable` : 심볼 테이블

주요 함수:

- `Compile()` : AST 노드를 바이트코드로 컴파일
- `emit()` : 명령어 생성 및 추가
- `Bytecode()` : 최종 바이트코드 반환

심볼 관리:

- `SymbolTable` : 변수/함수 심볼 관리
- 스코프별 심볼 정의 및 해결

5. 가상 머신 실행 (VM Execution) - vm/

주요 구조체:

- `VM` : 바이트코드 실행 엔진

- `stack` : 연산 스택
- `sp` : 스택 포인터
- `constants` : 상수 풀
- `globals` : 전역 변수
- `frames` : 함수 호출 프레임들

주요 함수:

- `Run()` : 바이트코드 실행 메인 루프
- `push()` , `pop()` : 스택 관리
- `executeBinaryOperation()` : 이항 연산 실행
- `executeComparison()` : 비교 연산 실행

프레임 관리:

- `Frame` : 함수 호출 컨텍스트
- 지역 변수와 명령어 포인터 관리

6. 바이트코드 시스템 - code/

Opcode 정의:

```
OpConstant, OpPop, OpAdd, OpSub, OpMul, OpDiv,

OpTrue, OpFalse, OpEqual, OpNotEqual, OpGreaterThan,

OpMinus, OpBang, OpJumpNotTruthy, OpJump, OpNull,

OpGetGlobal, OpSetGlobal, OpArray, OpHash, OpIndex,

OpCall, OpReturnValue, OpReturn, OpGetLocal, OpSetLocal,

OpGetBuiltin, OpClosure, OpGetFree
```

주요 함수:

- `Make()` : Opcode와 피연산자로 명령어 생성
- `Lookup()` : Opcode 정의 조회
- `ReadUint16()` : 16비트 피연산자 읽기

7. REPL 시스템 - repl/

주요 기능:

- 사용자 입력 받기
- 렉싱 → 파싱 → 컴파일 → VM 실행 파이프라인
- 결과 출력 및 에러 처리
- 전역 상태 유지 (변수, 함수 등)

데이터 흐름

```
소스 코드 (문자열)
↓
Lexer.NextToken() → 토큰 스트림
↓
Parser.ParseProgram() → AST
↓
┌─ Evaluator.Eval() → 직접 실행 (인터프리터)
└─ Compiler.Compile() → 바이트코드 → VM.Run() (컴파일러)
↓
```

주요 설계 패턴

1. **Visitor 패턴**: AST 노드 순회 및 처리
2. **Strategy 패턴**: 전위/중위 파싱 함수 등록
3. **Factory 패턴**: 객체 생성 및 토큰 생성
4. **Interpreter 패턴**: AST 직접 평가
5. **Stack Machine**: VM의 스택 기반 실행

테스트 구조

각 컴포넌트마다 `*_test.go` 파일이 있어 단위 테스트를 제공합니다:

- 렉서: 토큰화 정확성 테스트
- 파서: AST 생성 및 연산자 우선순위 테스트
- 평가기: 표현식 평가 및 환경 관리 테스트
- 컴파일러: 바이트코드 생성 테스트
- VM: 바이트코드 실행 테스트

이 구조 분석을 바탕으로 각 컴포넌트별 상세 문서를 작성할 예정입니다.